Tiny210 MT7601U USB Wi-Fi 驱动架构详解

分层结构 · USB 各层 · 802.11/WPA2 · 网络协议栈 · 调用链与实板日志

xv6 Tiny210（S5PV210, ARMv7-A）移植

2026 年 11 月

目录

1 概述 **3**

2 总体分层 **3**

3 硬件连接 **4**

4 软件架构 **5**

4.1 执行上下文 6

5 USB 各层架构 **7**

5.1 平台层：PHY 与控制器启动 7

5.2 HCD 层：QH/qTD 与 dummy 追加法 8

5.3 枚举与 Hub 8

5.4 USB core 与设备驱动接口 8

6 MT7601U 硬件层 **9**

6.1 probe 时的初始化 9

6.2 USB 上的帧格式 10

6.3 WCID 与密钥表 11

7 802.11 与 WPA2：网卡如何和路由器通信 **11**

7.1 连接状态机 11

7.2 各阶段帧 11

7.3 WPA2-PSK 密钥推导（wpa_crypto.c） 12
7.4 802.11 数据帧与 Ethernet 帧的转换 12

8 网络协议层 13

9 时序图 14

9.1 枚举与固件下载 14
9.2 连接与 WPA2 四次握手 15
9.3 DHCP 与 ARP 数据路径 16

10 函数调用链与日志对照 16

10.1 驱动注册与 probe 16
10.2 接收路径 17
10.3 连接：系统调用与状态机 17
10.4 发送路径（DHCP） 18
10.5 完整实板日志（成功的一次） 18

11 实板调试中的问题与所在层 19

1 概述

本文说明 Tiny210（S5PV210）上 xv6 的 USB Wi-Fi 链路：一个 MT7601U USB 网卡从插在 SDK 底板 USB-A Host 口开始，到用 WPA2 连上 TP-Link_B114 路由器、DHCP 拿到 IP 为止，每一层用的是哪个驱动、哪个函数，以及实板串口上对应打印哪一行日志。文中的日志都是 2026-11-14 在实板上抓到的真实输出。Markdown 版见 `doc/readme/tiny210/wifi-architecture.md`。

和 Linux 一样，整个链路按“谁只关心哪一层”切开：用户态只看到“连接 / DHCP / ping”；`net.c` 只看到一个能收发 Ethernet 帧的 `wlan0`；`mt7601u.c` 把 Ethernet 帧翻译成 802.11 帧并管理连接状态；`ehci-hcd.c` 只负责把字节按 USB 协议搬到设备的某个端点。

2 总体分层

用户程序 `init`（读 /etc/wifi.conf）→ `/bin/wifi`、`/bin/dhcp wlan0`、`/bin/ping`

系统调用 `usys.S`（r7= 号, svc 0）→ `sysnet.c`: `sys_wifi_connect`、`sys_net_dhcp`

网络协议栈 L3/L4 `net.c`: ARP · IPv4 · 路由 · UDP · ICMP · TCP · DHCP 客户端

网络设备层 `struct net_device` “wlan0”，`netdev_xmit_on` / `net_rx_dev`，NAPI，workqueue

802.11 SoftMAC L2 `mt7601u.c`: 扫描·认证·关联·WPA2 四次握手·802.11 Ethernet；`wpa_crypto.c`

USB 设备驱动 `mt7601u_driver`: id_table 148f:7601，probe，固件下载，eFuse，MAC/BBP/RF 初始化

USB core `usb/core/usb.c`: usb_bus，`usb_match`，`usb_device_register`，`struct usb_host_ops`

主机控制器 HCD `usb/host/ehci-hcd.c`: 异步调度 QH/qTD，dummy 追加，Hub，split，IRQ

平台/板级 `mach-s5pv210/devices.c`: s5p-ehci MEM 0xEC200000，IRQ 55，`host_phy_init`

硬件 USB PHY1 → USB2514 Hub → MT7601U（MCU+MAC+BBP+RF）→ 2.4 GHz → 路由器

图 1: Wi-Fi 链路分层

层	实现	与下一层的接口
用户程序	`user/wifi.c`, `dhcp.c`, `ping.c`	系统调用 `wifi_connect`、`net_dhcp`、`icmp_send`
系统调用	`kernel/sysnet.c`	`mt7601u_connect()`、`net_dhcp_dev()`
协议栈	`kernel/net.c`	`netdev_xmit_on()` / `net_rx_dev()`
网络设备	`register_netdev`、NAPI、`workqueue.c`	`net_device_ops.start_xmit`
SoftMAC	`mt7601u.c` + `wpa_crypto.c`	厂商 USB 寄存器请求 + bulk 帧 (TXWI/RXWI)
USB 设备驱动	`mt7601u.c` 的 `usb_driver`	`usb_host_ops`: control / bulk / bulk_rx_arm / bulk_rx_complete
USB core	`usb/core/usb.c`	`usb_device_register()`
HCD	`usb/host/ehci-hcd.c`	寄存器 + DMA 描述符
平台	`mach-s5pv210/devices.c`	platform device 资源

3 硬件连接

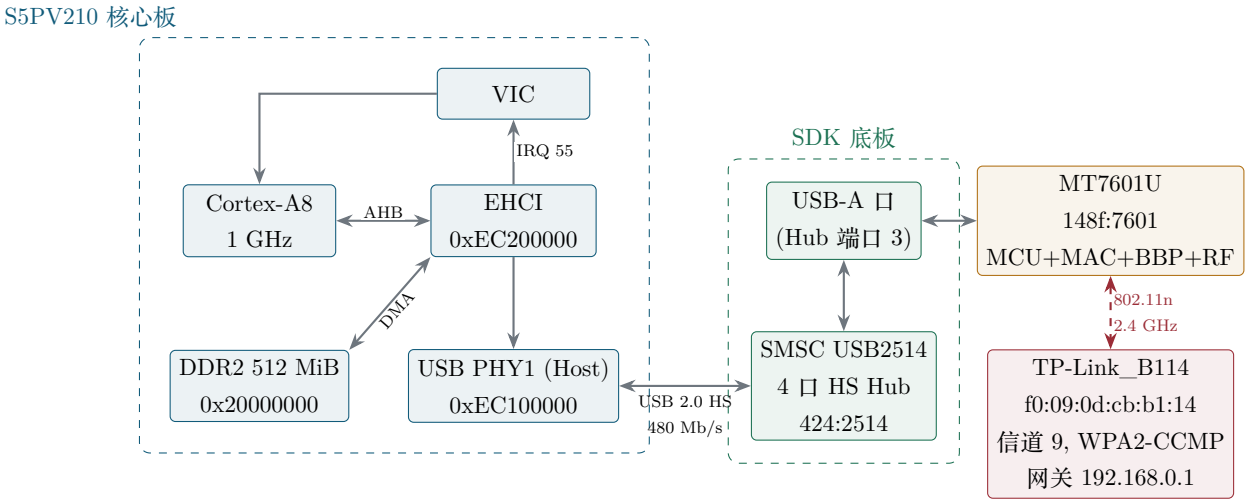


图 2: 硬件连接

- 只走 EHCI（USB-A Host 口），不走 OTG 口。OTG 是 DWC2 控制器（0xEC000000，IRQ 56），代码保留但默认不编（`S5PV210_USB_OTG=1` 才启用）。
- SoC Host 只有 1 个根端口，SDK 底板上的 USB2514 Hub 扩出 4 口，网卡在 Hub 端口 3。Hub 和网卡都是高速设备，不需要 split transaction。

- EHCI 是总线主设备，自己从 DDR 读 QH/qTD 描述符和数据。xv6 在 Tiny210 上关闭 D-cache，不需要 cache 维护，但 CPU 访存慢、会和 EHCI 抢 DDR 带宽（见第 11 节）。

端点	方向	用途（本驱动）
EP0	control	厂商请求：读写 32 位寄存器、eFuse、FCE 固件 DMA 控制
0x84 (IN#0)	bulk IN	接收数据：RXWI + 802.11 帧（4 个 16 KiB 缓冲循环）
0x85 (IN#1)	bulk IN	MCU 命令响应
0x08 (OUT#0)	bulk OUT	固件下载、MCU 命令，以及本驱动所有 802.11 发送帧
0x04–0x07, 0x09	bulk OUT	Linux 的 AC/管理队列；Tiny210 上 0x09 一开始就 NAK，不用

表 2: MT7601U USB 端点（日志 `bulk in 4,5/512 out 8/512`）

4 软件架构

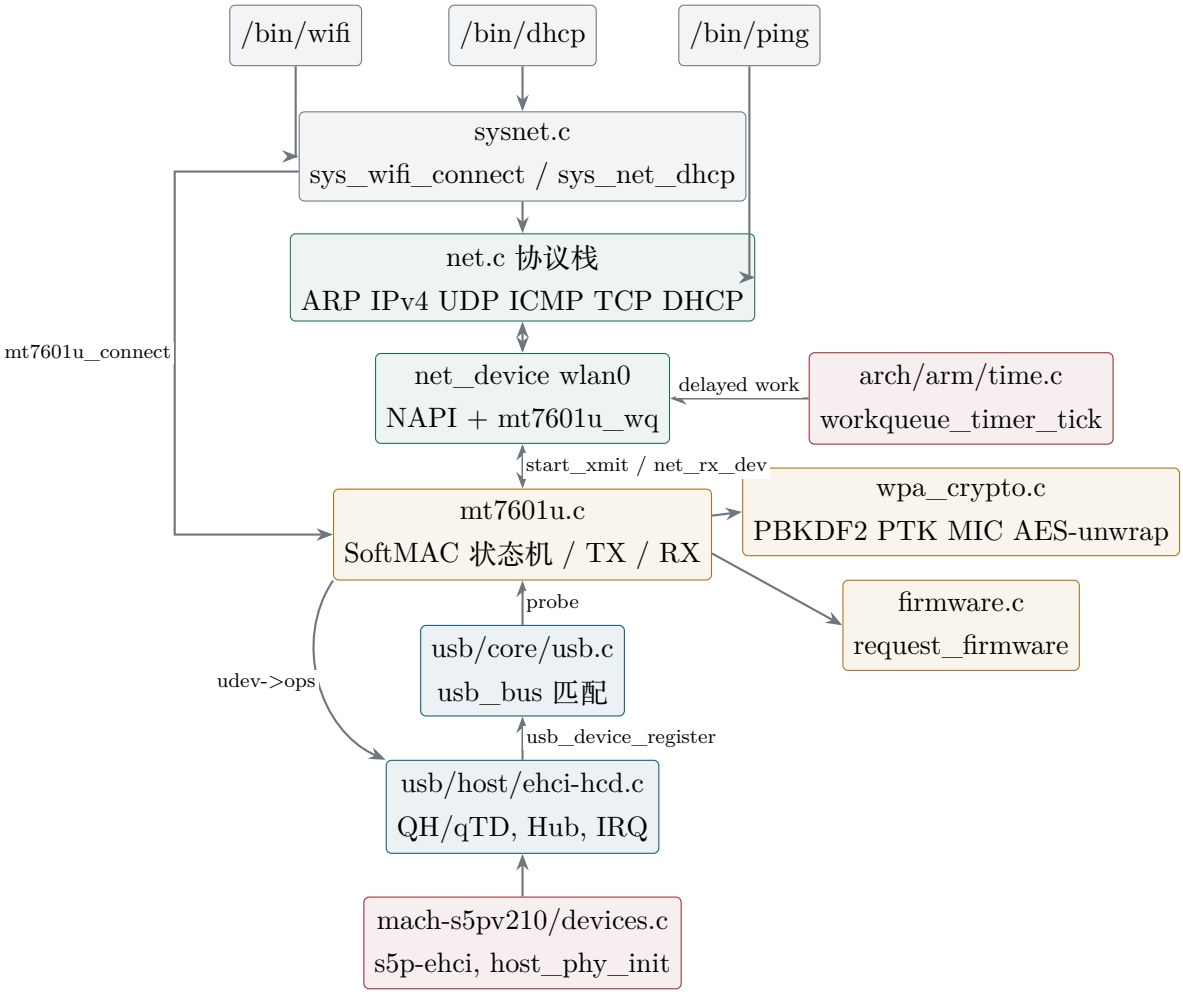


图 3: 软件架构与源文件

4.1 执行上下文

上下文	做什么
ehci-enum 内核线程	启动 EHCI、上电端口、枚举 Hub 与网卡；在此线程中调用 mt7601u_probe（固件下载、射频初始化）。要等几百毫秒的复位/上电，所以不放在 initcall 里
EHCI 中断(IRQ 55)	ehci_irq 清 USBSTS，调用 mt7601u_rx_irq() → napi_schedule()，只做排队
mt7601u_wq 线程	NAPI: mt7601u_napi_poll → mt7601u_service(1,0) 处理接收；state_work: mt7601u_state_worker → mt7601u_service(0,1) 推进扫描/认证/关联/超时
时钟中断 100 Hz	arm32_timer_tick → workqueue_timer_tick(): 到期 delayed work 入队
用户进程	mt7601u_connect 设置目标后 sleep(&ticks) 等 handshake_done; dhcp/ping 在系统调用里发帧、等回包

5 USB 各层架构

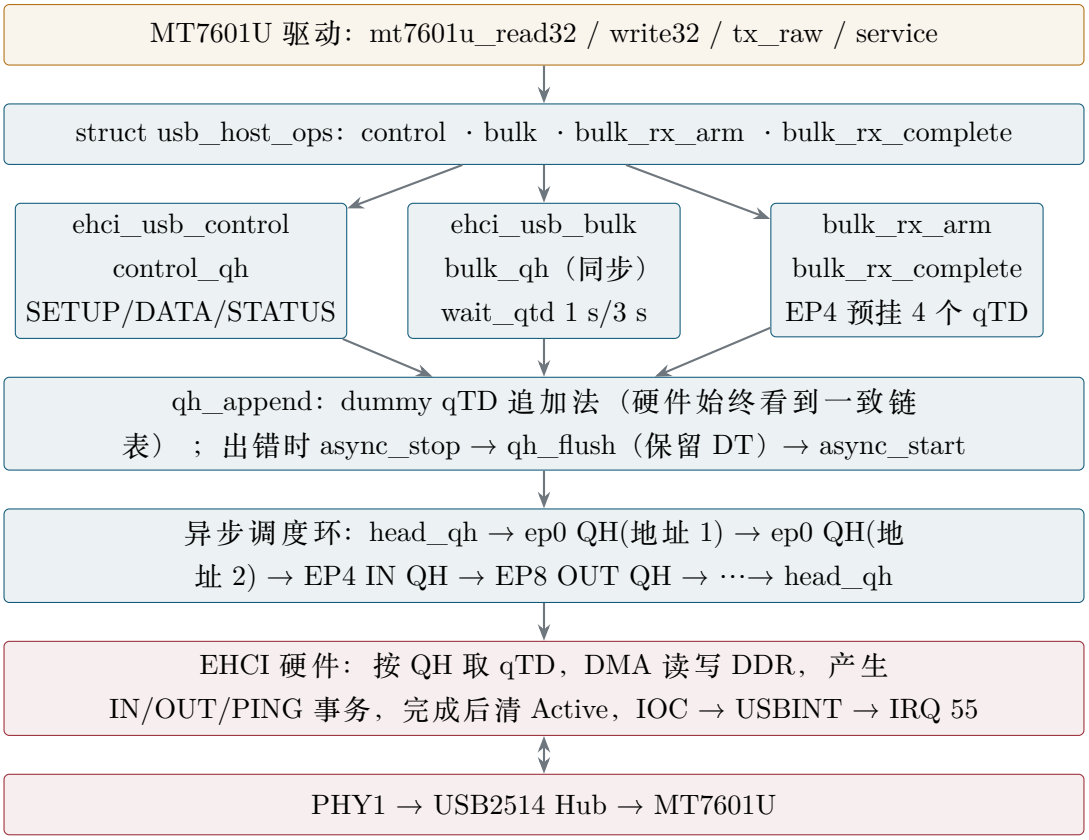


图 4: USB 软件分层

5.1 平台层：PHY 与控制器启动

`host_phy_init()`（参考 Linux `phy-s5pv210-usb2.c`）：`CLK_GATE_IP1`(0xE0100464) bit17 打开 USBHOST 时钟；`USB_PHY_CONTROL`(0xE010E80C) bit1 解除 PHY1 隔离；PHY 寄存器块 0xEC100000 中 `UPHYCLK` 选 24 MHz，`UPHYPWR` 清 bit6/7 使 PHY1 上电，`URSTCON` bit3/4 脉冲复位 PHY1 和 Host link。

`ehci_start()`：

1. `phy_init()`；CAPLENGTH 得到操作寄存器基址，HCSPARAMS 得到端口数（1）。
2. Samsung 私有寄存器 INSNREG00（burst 使能），**INSNREG01 = 0x00800080**（OUT/IN 阈值均为 512 字节）。
3. 停止、复位控制器；建立异步环头 QH（H 位，halted overlay，指向自己）。
4. `ASYNCLIST = head_qh`，`USBINTR = USBINT|ERR`，`USBCMD = RUN|ASE|ITC(1)`，`CONFIGFLAG = 1`；端口上电，等 100 ms。

```
ehci: s5p-ehci at -13e00000, enumeration deferred to ehci-enum
ehci: 1 port(s), running, async schedule on
```

5.2 HCD 层：QH/qTD 与 dummy 追加法

EHCI 异步调度是一个 QH（每端点一个）组成的环，每个 QH 下挂一串 qTD，每个 qTD 最多描述 5 个 4 KiB 页。

- `qh_create(addr, ep, speed, mps, control, hub, port)`：填 epchar（地址、端点、速度、最大包长）、epcap（Hub 地址/端口），挂入环；初始挂一个不活动的 dummy qTD。
- `qh_append(q, first, last)`：把新传输第一个 qTD 的内容拷进当前 dummy，在尾部接新 dummy；先激活后续 qTD，最后才置第一个的 Active。无需停异步调度。
- `wait_qtd`：轮询最后一个 qTD 的 Active（IN 1 s，OUT 3 s，OUT 期间设备可 NAK/PING）。
- `qh_flush`：超时/出错时丢弃排队的 qTD，QH 重新指向 dummy，**保留 data toggle**。
- `ehci_usb_bulk_rx_arm`:给 EP4 预挂带 IOC 的 IN qTD;`ehci_usb_bulk_rx_complete` 从 `q->head` 取已完成的 qTD。
- `ehci_irq`：USBSTS 的 USBINT/ERR 置位时调弱符号 `mt7601u_rx_irq()`。

5.3 枚举与 Hub

`ehci_enum_thread` → `ehci_start` → `root_ports()`：根端口连接 → 端口复位 → `enumerate(HS, 0, 0)`：地址 0 读设备描述符前 8 字节 → `SET_ADDRESS` → 读设备/配置描述符 → `parse_config` 记录端点 → `SET_CONFIGURATION`。类 9（Hub）→ `hub_ports()`：读 Hub 描述符，各口 `PORT_POWER`，有设备则 `PORT_RESET` 并 `enumerate(HS, 1, 3)`；非 Hub → `usb_device_register()` → usb 总线遍历驱动 `id_table` → `mt7601u_probe()`。

```
ehci: port 0: high-speed device connected
ehci: high-speed device 424:2514 class 9 at address 1 (hub 0 port 0)
ehci:   bulk in 0,0/0 out 0/0 intr in 1
ehci: hub 1: 4 ports
ehci: high-speed device 148f:7601 class 255 at address 2 (hub 1 port 3)
ehci:   bulk in 4,5/512 out 8/512 intr in 0
```

5.4 USB core 与设备驱动接口

```
struct usb_host_ops {                       /* kernel/usb.h */
  int (*control)(struct usb_device*, uint8 type, uint8 req, uint16 value,
                 uint16 index, void *data, uint16 len);
  int (*bulk)(struct usb_device*, int ep, int in, void *data, int len);
  int (*bulk_rx_arm)(struct usb_device*, int ep, void *data, int len);
  int (*bulk_rx_complete)(struct usb_device*, int ep, void **data);
};
```

MT7601U 驱动不知道 EHCI 的存在，只调用 `udev->ops->...`；同一份驱动也能挂在 DWC2（OTG）上。

6 MT7601U 硬件层

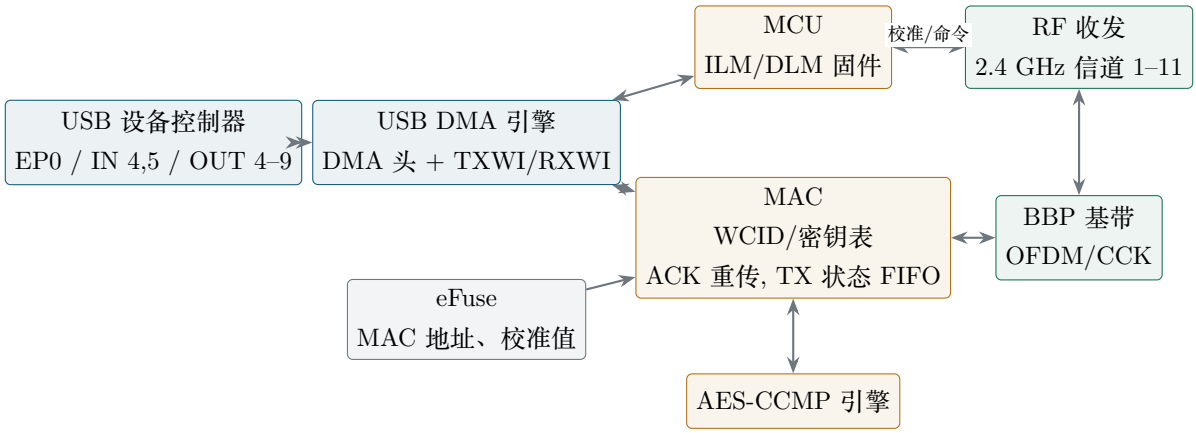


图 5: MT7601U 芯片内部

6.1 probe 时的初始化

#	函数	做什么	日志
1	`read32` (ASIC_VERSION)	EP0 厂商请求读寄存器，确认 0x7601	`ASIC=76010001`
2	`validate_firmware`	`request_firmware`: 读 /boot/mt7601u.bin，检查 ILM/DLM	`firmware v0.1.0 build=7640`
3	`load_firmware` → `dma_fw_range` → `dma_fw_chunk`	每块 14 KiB: EP0 `WRITE_FCE` 写 DMA 地址/长度，EP8 送 TXINFO+ 数据，`CPU_DESC_IDX`+1，轮询 `COM_REG1` bit31；ILM 后 DLM，启动 MCU	`ILM firmware upload ...`，`MCU firmware running`
4	`read_eeprom` → `efuse_read_block`	eFuse 读 MAC、频偏、温度补偿	`mac=90:de:80:14:95:94`
5	`basic_hw_init`	WLAN 时钟/PLL、BBP 就绪、RF 引导	`bootstrap ready; rx-ep=4`
6	`mcu_channel_init`	MCU 命令走 EP8、响应走 EP5	`tx-ep=8 resp-ep=5`

#	函数	做什么	日志
7	`full_hw_init`	MAC 44 项 / BBP 59 项 / RF 138 项，`calibrate`	`RF calibration complete`
8	`set_channel(1)`	从信道 1 开始扫描	
9	`bulk_rx_arm` ×4	EP4 挂 4 个 16 KiB 接收缓冲	`EP4 receive uses ...`
10	`queue_delayed_work`	启动状态机，每 0.5 s 换信道	`SoftMAC initialization pending`

6.2 USB 上的帧格式

MT7601U 是 SoftMAC：802.11 帧由主机驱动组好，芯片只负责 PHY 头、调制、ACK、重传和可选的加解密。

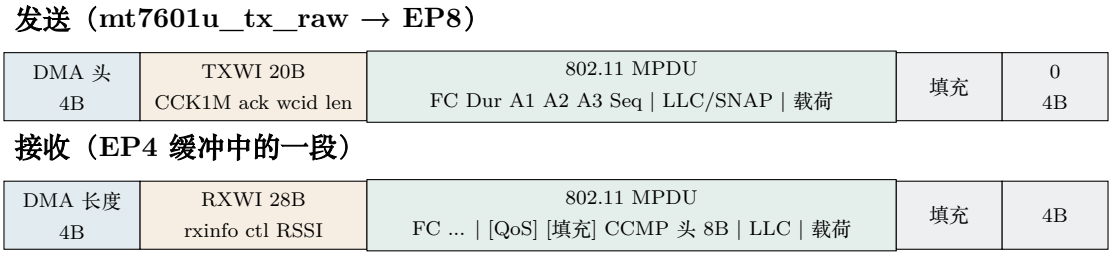


图 6: USB 帧封装

- `MT_TXD_PKT_INFO_80211`：载荷是原始 802.11 帧。`WIV`（Without IV）置位表示不做硬件加密。管理帧和握手前 EAPOL 置 WIV；**握手后数据帧不置 WIV，wcid=1**，MAC 用 WCID 1 的 TK 做 CCMP，自动插入 CCMP 头和 MIC。
- 发完后 `mt7601u_tx_stat_drain` 读空 TX_STAT_FIFO。
- RXINFO：bit8 CRC 错、bit9 ICV 错、bit10 MIC 错、bit14 L2 填充、**bit16 DECRYPT**、bit4 单播给我、bit5 组播、bit6 广播、bit7 本 BSS。实板 `rxinfo=1520e2` = DECRYPT+ 本 BSS+ 广播 + 组播 + 数据；`112092` = DECRYPT+ 本 BSS+ 单播给我（ARP 回复）。

6.3　WCID 与密钥表

寄存器区	地址	内容（mt7601u_install_ccmp_keys）
单播密钥表	0x8000 + WCID×32	WCID 1（AP）的 TK = PTK[32..47]
IV/EIV 表	0xA000 + WCID×8	初始 PN，KeyID 字节 0x20（ExtIV）
WCID 属性	0xA800 + WCID×4	cipher=4（AES-CCMP），pairwise=1
共享（组）密钥	0xAC00 + idx×32	GTK（从 M3 解出）
共享密钥模式	0xB000	每个 key index 4 位，填 4

7　802.11 与 WPA2：网卡如何和路由器通信

7.1　连接状态机

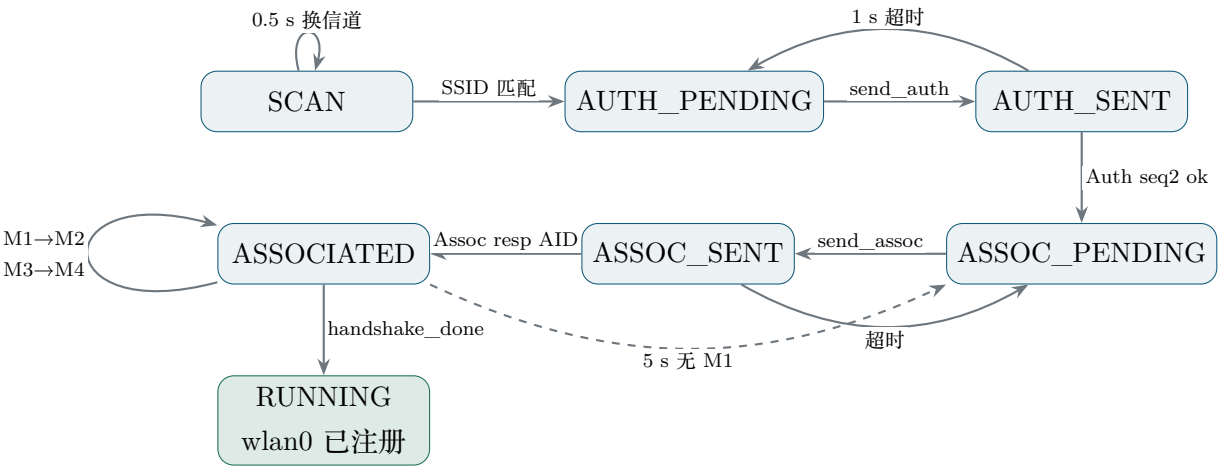


图 7: mt7601u 连接状态机（mt7601u_service）

时间常量（jiffy=10 ms）：扫描驻留 50（0.5 s），响应超时 100（1 s），WPA 看门狗 500（5 s），认证/关联最多重试 3 次。状态机完全靠 delayed work 推进，所以必须有 workqueue_timer_tick()。

7.2　各阶段帧

阶段	方向	802.11 帧	驱动函数	关键字段
扫描	AP→STA	Beacon (FC 0x0080)	rx_mgmt	SSID、DS 参数（信道）、RSN IE

阶段	方向	802.11 帧	驱动函数	关键字段
认证	STA→AP	Authentication (0x00B0)	`send_auth`	Open System, seq 1
	AP→STA	Authentication	`rx_mgmt`	seq 2, status 0
关联	STA→AP	Assoc Request (0x0000)	`send_assoc`	SSID、速率、RSN IE 22 字节
	AP→STA	Assoc Response (0x0010)	`rx_mgmt`	status 0, AID 1
M1	AP→STA	Data+LLC+EAPOL-Key (0x888e)	`rx_data`→`rx_eapol`	ANonce, key-info 0x008a
M2	STA→AP	Data（未加密）	`send_m2`	SNonce、RSN IE、MIC
M3	AP→STA	Data	`rx_eapol`	MIC、Install、加密 GTK
M4	STA→AP	Data	`send_m4`	MIC
数据	双向	Data, Protected, CCMP	`send_llc`/`rx_data`	ToDS/FromDS、CCMP 头

7.3 WPA2-PSK 密钥推导（wpa_crypto.c）

```
PMK    = PBKDF2-HMAC-SHA1(passphrase, SSID, 4096, 32)       wpa_pbkdf2     (mt7601u_connect)
SNonce = PRF(PMK, ANonce, counter)                           wpa_make_snonce (收到 M1)
PTK    = PRF-512(PMK, "Pairwise key expansion",
                 min(AA,SPA)|max(AA,SPA)|min(AN,SN)|max(AN,SN))   wpa_derive_ptk
  PTK[0..15]  = KCK -> EAPOL MIC (HMAC-SHA1 截 16 字节)      wpa_eapol_mic
  PTK[16..31] = KEK -> 解 M3 中的 GTK (AES Key Wrap)        wpa_aes_unwrap
  PTK[32..47] = TK  -> 写入芯片 WCID 1 密钥表，硬件 CCMP
GTK    = AES-Unwrap(KEK, M3 Key Data) 中 00-0f-ac type 1 的 KDE -> 共享密钥表
```

7.4 802.11 数据帧与 Ethernet 帧的转换

```
发送 mt7601u_send_llc:
 Ethernet | DA | SA | type | payload |
 802.11   | FC=0x4108 Data,ToDS,Protected | Dur | A1=BSSID | A2= 本机 | A3=DA | Seq |
          | AA AA 03 00 00 00 | type | payload |      (CCMP 头/MIC 由硬件插入)
接收 mt7601u_rx_data:
 802.11   | FC=0x4208 Data,FromDS,Protected | Dur | A1=DA | A2=BSSID | A3=SA | Seq |
          | [QoS 2] | [L2 填充 2] | CCMP 头 8 | AA AA 03 00 00 00 | type | payload |
```

```
Ethernet | DA=A1 | SA=A3 | type | payload |  -> net_rx_dev(&wlan0)
```

`type == 0x888e`（EAPOL）交给 `mt7601u_rx_eapol`，其它在握手完成后交给协议栈。

8 网络协议层

层	协议	实现函数（net.c）	本链路中的作用
L2 设备	net_device	`register_netdev`, `netdev_find`, `netdev_xmit_on`, `net_rx_dev`	wlan0 对上层就是一块以太网卡
L2	ARP	`arp_request`, `arp_rx`, `arp_lookup`	解析网关 192.168.0.1 的 MAC（即 AP BSSID）
L3	IPv4/路由	`net_output_path`, `route_lookup`, `checksum`	选接口与下一跳
L4	UDP	`net_udp_send`, `udp_rx`	DHCP 67/68、DNS
L4	ICMP	`net_icmp_send`, `icmp_rx`	ping
L4	TCP	`tcp_rx`, `tcp_send_*`	socket 程序
应用	DHCP	`net_dhcp_dev`, `dhcp_send`, `dhcp_rx`, `dhcp_wait`	DISCOVER → OFFER → REQUEST → ACK

`net_rx_dev` 分发：type 0x0806 → `arp_rx`；0x0800 → 校验 IP 头 → proto 1 `icmp_rx`、6 `tcp_rx`、17 且源 67/目的 68 → `dhcp_rx`，再交 `udp_rx`。

```
DISCOVER 0.0.0.0:68 -> 255.255.255.255:67  dst ff:ff:ff:ff:ff:ff  (data tx type=800 len=342)
OFFER    192.168.0.1:67 -> 广播             GTK 解密             (rxinfo=1520e2)
REQUEST  广播, 选项 50=192.168.0.219, 54= 服务器
ACK      -> 设置 ip_addr、网关、DNS, 加默认路由
ARP      who-has 192.168.0.1 (data tx type=806 len=42) -> reply 单播, PTK 解密 (rxinfo=112092)
```

9 时序图

9.1 枚举与固件下载

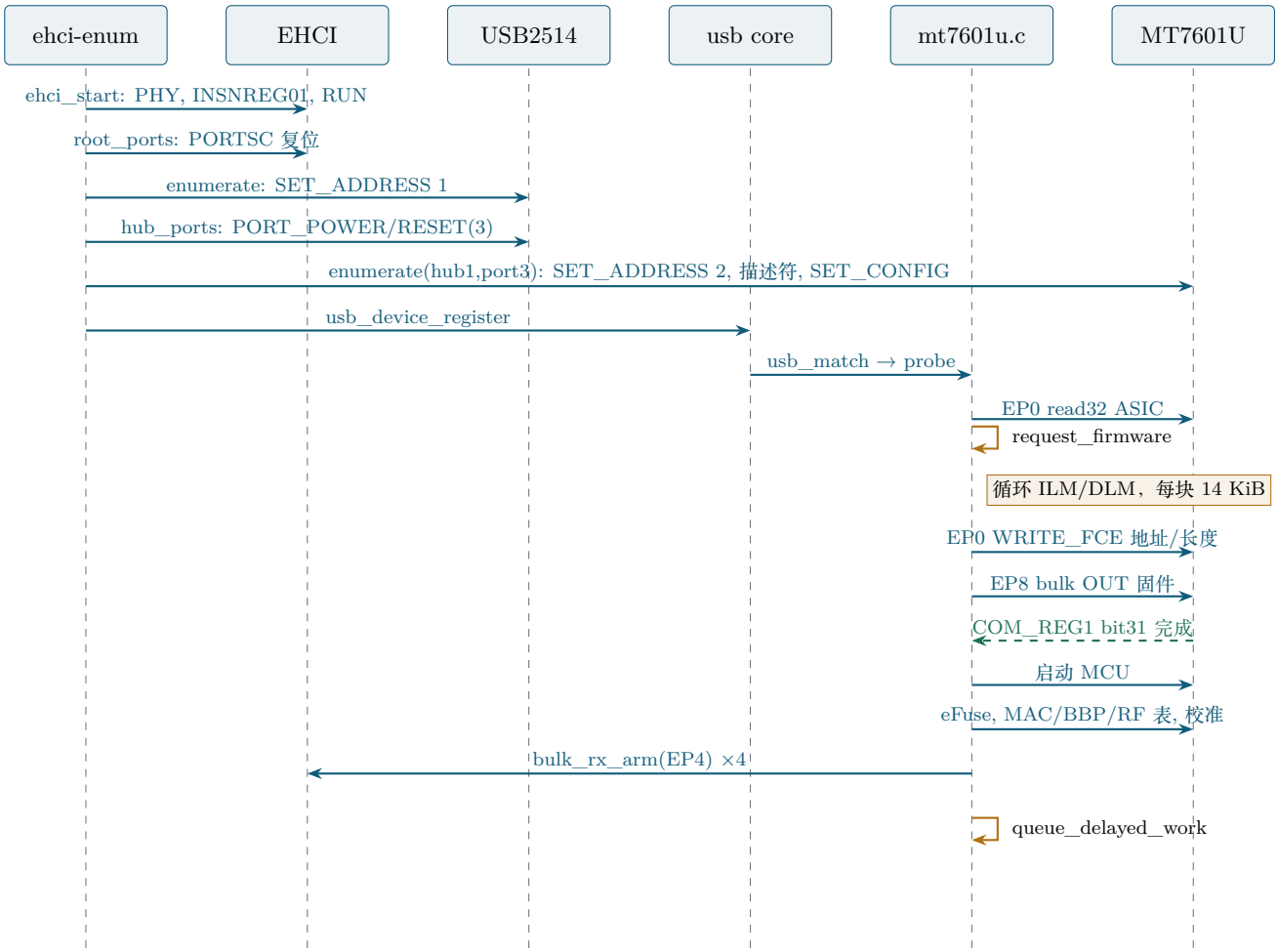


图 8: 上电枚举与固件下载

9.2 连接与 WPA2 四次握手

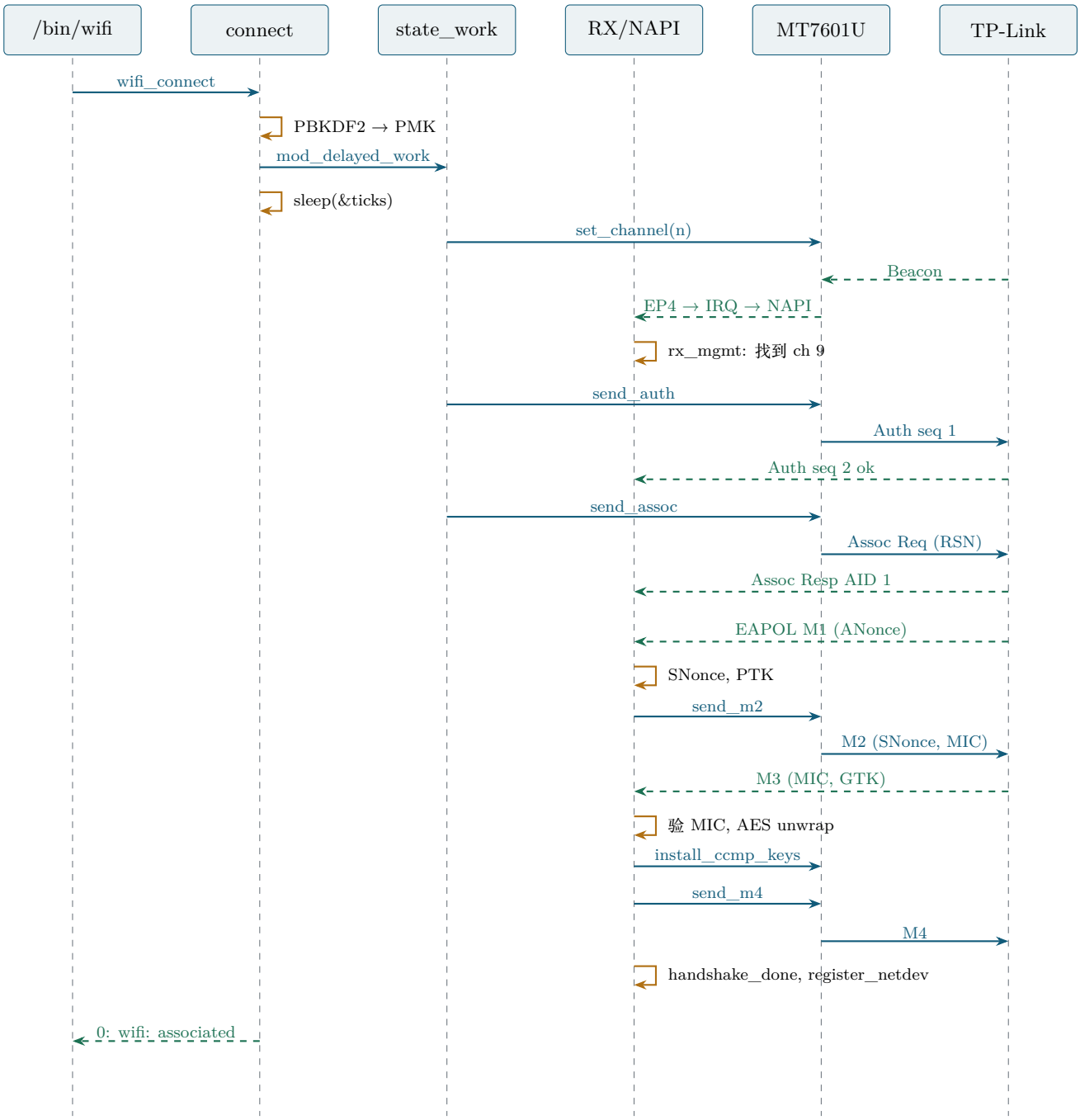


图 9: 扫描、认证、关联与四次握手

9.3 DHCP 与 ARP 数据路径

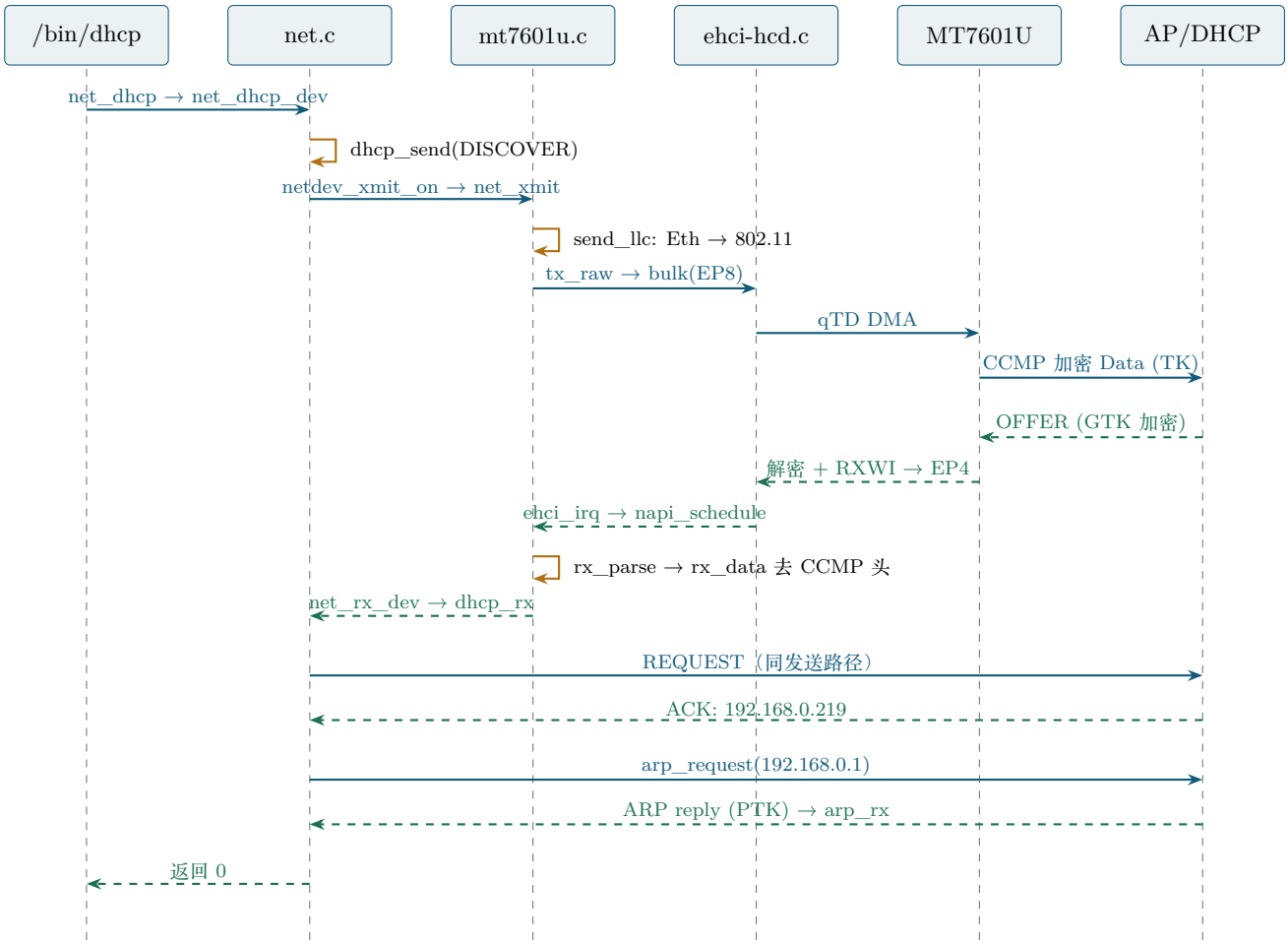


图 10: DHCP 与 ARP

10 函数调用链与日志对照

10.1 驱动注册与 probe

```
do_initcalls()
 |- mt7601u_driver_init() -> usb_register_driver(&mt7601u_driver)   id_table 148f:7601...
 `- ehci_probe("s5p-ehci") -> kthread ehci_enum_thread()
     |- ehci_start(): host_phy_init(), INSNREG01=0x800080, RUN, ASE
     `- root_ports()              "ehci: port 0: high-speed device connected"
         `- enumerate(HS,0,0)     "ehci: high-speed device 424:2514 class 9 at address 1"
             `- hub_ports()       "ehci: hub 1: 4 ports"
                 `- enumerate(HS,1,3)
                      "ehci: high-speed device 148f:7601 class 255 at address 2"
                     |- parse_config()  "ehci:   bulk in 4,5/512 out 8/512 intr in 0"
                     `- usb_device_register() -> usb_match() -> usb_probe()
                         `- mt7601u_probe()   (见下)

mt7601u_probe(udev)
```

```
|- mt7601u_read32() -> ops->control -> ehci_usb_control -> control_qh
|        "mt7601u: USB device 148f:7601 ASIC=76010001 MAC=76010500"
|- mt7601u_validate_firmware() -> request_firmware -> FAT32 /mt7601u.bin
|        "mt7601u: firmware v0.1.0 build=7640 size=45412 validated"
|- mt7601u_load_firmware() -> dma_fw_range() -> dma_fw_chunk()
|   `- ops->bulk(EP8) -> ehci_usb_bulk -> bulk_qh -> qh_append -> wait_qtd
|        "mt7601u: ILM firmware upload 14336/45316" ... "MCU firmware running"
|- mt7601u_read_eeprom() -> efuse_read_block()
|        "mt7601u: permanent mac=90:de:80:14:95:94"
|- basic_hw_init() / mcu_channel_init() / full_hw_init() -> calibrate()
|        "mt7601u: RF calibration complete; bulk endpoints ready"
|- set_channel(1), netif_napi_add(), init_delayed_work(state_work)
|- ehci_usb_bulk_rx_arm(EP4) x4
`- queue_delayed_work(state_work, 50)
         "mt7601u: transport and MCU ready; SoftMAC initialization pending"
```

10.2 接收路径

```
EHCI 完成 EP4 IN qTD (IOC)
 `- IRQ 55 -> VIC -> ehci_irq()                          清 USBSTS
    `- mt7601u_rx_irq() -> napi_schedule() -> queue_work(mt7601u_wq)
       ... 工作队列线程 ...
       napi_workfn() -> mt7601u_napi_poll(budget)
        `- mt7601u_service(1, 0)
            |- ops->bulk_rx_complete(EP4) = ehci_usb_bulk_rx_complete
            |- mt7601u_rx_parse(buf, len)               逐段: DMA 长度 + RXWI
            |   |- 管理帧 -> mt7601u_rx_mgmt()          Beacon / Auth / Assoc resp
            |   `- 数据帧 -> mt7601u_rx_data(frame, len, rxinfo)
            |        |- Protected 且 DECRYPT -> 跳过 CCMP 头 8 字节
            |        |- type 0x888e -> mt7601u_rx_eapol()   M1 / M3
            |        `- 其他 -> 组成 Ethernet 帧放入 dev->net_rx
            |- ops->bulk_rx_arm(EP4, 同一缓冲)          重新挂回接收环
            `- net_rx_dev(&wlan0) -> arp_rx / icmp_rx / dhcp_rx / udp_rx
                   "arp: dev=wlan0 gateway 192.168.0.1 is f0:9:d:cb:b1:14"
```

10.3 连接：系统调用与状态机

```
init -> fork -> exec /bin/wifi (读 /etc/wifi.conf)
 `- wifi_connect(ssid, psk)   [usys.S: r7=SYS_wifi_connect; svc 0]
    `- sys_wifi_connect()                          "wifi: connecting to TP-Link_B114"
        `- mt7601u_connect(ssid, psk, 30000)
            |- wpa_pbkdf2(psk, ssid, pmk)
            |- link_state=SCAN; mod_delayed_work()  "mt7601u: joining TP-Link_B114"
            `- sleep(&ticks) until handshake_done

arm32_timer_tick -> workqueue_timer_tick -> state_work 到期
 `- mt7601u_state_worker() -> mt7601u_service(0, 1)
```

```
     |- SCAN 超时  -> mt7601u_set_channel(next)
     |- AUTH_PENDING  -> send_auth -> tx_mgmt -> tx_raw(wcid 0xff, WIV)
     |                                    "mt7601u: authentication request sent"
     `- ASSOC_PENDING -> send_assoc -> tx_mgmt -> tx_raw
                                        "mt7601u: association request sent rsn=22 bytes"
mt7601u_rx_mgmt():
 |- Beacon SSID 匹配  "mt7601u: target TP-Link_B114 found bssid=f0:9:d:cb:b1:14 channel=9 WPA2"
 |- Auth seq2 ok      "mt7601u: Open-System authentication accepted"
 `- Assoc resp ok     "mt7601u: 802.11 associated aid=1; WPA2 handshake pending"
mt7601u_rx_eapol():
 |- M1: wpa_make_snonce, wpa_derive_ptk   "mt7601u: received WPA2 EAPOL M1 key-info=8a"
 |   `- send_m2 -> wpa_eapol_mic -> send_llc(0x888e, enc=0) -> tx_raw(WCID 1, WIV)
 |                                         "mt7601u: sent WPA2 EAPOL M2 replay=0001"
 `- M3: 校验 MIC -> wpa_aes_unwrap(KEK) -> GTK
     |- install_ccmp_keys()   TK->0x8000+32, IV->0xA000+8, attr->0xA800+4, GTK->0xAC00
     |- send_m4 -> send_llc(enc=1)          "mt7601u: sent WPA2 EAPOL M4"
     `- handshake_done=1                    "mt7601u: WPA2 four-way handshake complete"
mt7601u_service -> register_netdev(&wlan0)  "mt7601u: wlan0 registered mac=90:de:80:14:95:94"
mt7601u_connect 返回 0                      "wifi: associated"
```

10.4 发送路径（DHCP）

```
init -> /bin/dhcp wlan0
 `- net_dhcp("wlan0") -> sys_net_dhcp -> netdev_find -> net_dhcp_dev(dev)
     `- dhcp_send(DISCOVER)
         `- netdev_xmit_on(dev, frame, 342) -> ops->start_xmit = mt7601u_net_xmit()
             `- mt7601u_send_llc(ff:ff.., 0x0800, enc=1)        FC=0x4108
                 `- mt7601u_tx_raw(frame, n, MT_WCID_AP, hw_iv=1)  无 WIV -> 硬件 CCMP
                     |- ops->bulk(EP8) -> ehci_usb_bulk -> bulk_qh -> qh_append -> wait_qtd(3 s)
                     `- mt7601u_tx_stat_drain()
                     "mt7601u: data tx type=800 len=342 dst=ff:ff:ff:ff:ff:ff -> 0"
 OFFER (接收路径)    "mt7601u: data rx fc=4208 len=616 rxinfo=1520e2 a1=ff:ff:ff:ff:ff:ff"
 REQUEST -> ACK      "dhcp: dev=wlan0 address=192.168.0.219 gateway=192.168.0.1 dns=192.168.0.1"
 arp_request(gw)     "mt7601u: data tx type=806 len=42 dst=ff:ff:ff:ff:ff:ff -> 0"
 arp_rx(reply)       "mt7601u: data rx fc=4a08 len=68 rxinfo=112092 a1=90:de:80:14:95:94"
                     "arp: dev=wlan0 gateway 192.168.0.1 is f0:9:d:cb:b1:14"
```

10.5 完整实板日志（成功的一次）

```
mt7601u: firmware v0.1.0 build=7640 size=45412 validated
mt7601u: ILM firmware upload 14336/45316
mt7601u: ILM firmware upload 45316/45316
mt7601u: MCU firmware running
mt7601u: EEPROM ver=d fae=0 country=ff freq=96 rssi=0,0 temp=-7 lna=0
mt7601u: permanent mac=90:de:80:14:95:94
mt7601u: MAC/BBP/RF bootstrap ready; rx-ep=4 data, 5 MCU
mt7601u: MCU command channel ready tx-ep=8 resp-ep=5
```

```
mt7601u: MAC register table initialized (44 entries)
mt7601u: BBP register table initialized (59 entries)
mt7601u: RF register table initialized (138 entries)
mt7601u: RF calibration complete; bulk endpoints ready
mt7601u: transport and MCU ready; SoftMAC initialization pending
mt7601u: joining TP-Link_B114
mt7601u: target TP-Link_B114 found bssid=f0:9:d:cb:b1:14 channel=9 WPA2
mt7601u: authentication request sent
mt7601u: Open-System authentication accepted
mt7601u: association request sent rsn=22 bytes
mt7601u: 802.11 associated aid=1; WPA2 handshake pending
mt7601u: received WPA2 EAPOL M1 key-info=8a
mt7601u: sent WPA2 EAPOL M2 replay=0001
mt7601u: sent WPA2 EAPOL M4
mt7601u: WPA2 four-way handshake complete
mt7601u: wlan0 registered mac=90:de:80:14:95:94
wifi: associated
mt7601u: data tx type=800 len=342 dst=ff:ff:ff:ff:ff:ff -> 0
mt7601u: rx so far: usb buffers 244, frames 244 (mgmt 236 data 8), dropped 0
mt7601u: data rx fc=4208 len=616 rxinfo=1520e2 a1=ff:ff:ff:ff:ff:ff
dhcp: dev=wlan0 address=192.168.0.219 gateway=192.168.0.1 dns=192.168.0.1
mt7601u: data tx type=806 len=42 dst=ff:ff:ff:ff:ff:ff -> 0
arp: dev=wlan0 gateway 192.168.0.1 is f0:9:d:cb:b1:14
mt7601u: data rx fc=4a08 len=68 rxinfo=112092 a1=90:de:80:14:95:94
```

日志字段：`key-info=8a` 即 0x008a = 版本 2 + Pairwise + ACK（M1）；`fc=4208` = Data+FromDS+Protected，`6208` 多 More Data，`4a08` 多 Retry；`rxinfo=1520e2` 的 bit16 DECRYPT=1；`dropped 0` 表示没有 CRC/ICV/MIC 错误帧；`-> 0` 表示 EP8 bulk OUT 整帧送达。

11 实板调试中的问题与所在层

层	现象	根因	修复
HCD	固件下载 `bulk ep8 out 14344 bytes failed`，qTD 0xa0	CPU 不开 cache 占满 DDR，EHCI 默认阈值下发包欠载	INSNREG01=0x800080 + 开 I-cache（f9bfbc6）
内核定时	扫描停在信道 1，30 s 无握手	ARM32 时钟中断未调 `workqueue_timer_tick()`	在 `arm32_timer_tick` 中调用（0bd029e）
HCD	握手后 EP8 永远 NAK	`qh_flush` 清零 data toggle，设备把新包当重传丢弃	保留 `TOK_DT`（f649f28）
设备驱动	改 EP 0x09 后首帧即 NAK	OUT#5 在本驱动初始化方式下不可用	回 EP 0x08 + 读空 TX 状态 FIFO（5ac8c59）

层	现象	根因	修复
802.11	握手成功但 `dhcp: no offer`	硬件解密后仍保留 Protected 位和 8 字节 CCMP 头，LLC 校验失败	DECRYPT 时跳过 8 字节（b2326c2）→ DHCP 成功

尚未完成：实板 ping 外网确认；去掉或开关化 `data tx`、`data rx`、`rx so far`、`rx drop` 调试打印。